



PRATHYUSHA ENGINEERING COLLEGE

An Autonomous Institution

INDUSTRIAL INTERNSHIP REPORT ON SMART IRRIGATION SYSTEM

ESP32-Based Embedded Systems and IoT Project

Prepared as a Project Report
Academic / Internship Project

Prepared by
K.Prudhvi Raj

Prathyusha Engineering College
An Autonomous Institution

Final Project - Free Summer Internship in Embedded and IoT - 2026
upskill Campus | The IoT Academy | UniConverge Technologies Pvt Ltd (UCT)

Date of Submission: 30 August 2026

Wokwi Project: <https://wokwi.com/projects/473414532081154049>

Executive Summary

This report presents the Smart Irrigation System developed as the final project for the Free Summer Internship in Embedded and IoT - 2026. The report follows the structure and submission expectations shown in the Upskill Campus / UCT sample internship report.

The project uses an ESP32 development board, an SSD1306 128 x 64 OLED display, a relay module and a potentiometer-based input in the Wokwi simulation. The program maintains a simulated soil-moisture value, compares it with a threshold of 2000 and changes the relay state accordingly.

When the value is below 2000, the program treats the soil as dry and turns the relay ON. When the value is 2000 or above, the soil is treated as wet and the relay is turned OFF.

A key implementation detail is that the supplied program currently changes the soil value in software rather than reading the potentiometer on GPIO 34. The Wokwi circuit nevertheless includes the potentiometer and connects its signal to GPIO 34. The present submission therefore demonstrates the control logic and simulation; physical sensor integration is identified as future work.

The internship improved practical understanding of ESP32 programming, GPIO control, OLED/I2C interfacing, relay control, simulation, testing, debugging and technical documentation.

TABLE OF CONTENTS

Section	Title
1	Preface
2	Introduction
2.1	About UniConverge Technologies Pvt Ltd
2.2	About upskill Campus (USC)
2.3	The IoT Academy
2.4	Objectives of this Internship Program
2.5	Reference
2.6	Glossary
3	Problem Statement
4	Existing and Proposed Solution
4.1	Code Submission (GitHub Link)
4.2	Report Submission (GitHub Link)
5	Proposed Design / Model
5.1	High Level Diagram
5.2	Low Level Diagram
5.3	Interfaces
6	Performance Test
6.1	Test Plan / Test Cases
6.2	Test Procedure
6.3	Performance Outcome
7	My Learnings
8	Future Work Scope
Conclusion	
Appendix	Complete Source Code

1 Preface

The six-week Industrial Internship in Embedded Systems and IoT provided an opportunity to connect classroom learning with practical embedded and IoT work. The internship sample emphasizes practical industry experience, real-world problem solving, improved job prospects, understanding the field and its applications, and personal growth.

During this project, I worked on a Smart Irrigation System using an ESP32 and Wokwi simulation. The main idea was to automate irrigation decisions using a soil-condition value, display the status on an OLED and control an irrigation output through a relay.

The project work involved understanding the circuit, preparing the ESP32 program, interfacing the OLED through I2C, implementing threshold logic, observing serial output, testing dry and wet conditions and documenting the result. Wokwi made it possible to repeat these tests safely without a physical pump.

The project also helped me understand the importance of practical constraints. Because the current project is a simulation and the soil value is generated in software, real sensor accuracy, pump performance, power consumption and field durability could not be measured.

I am thankful to the internship organizers, mentors and faculty members who supported the learning process. The project also provides a useful foundation for a future version with a real soil-moisture sensor, Wi-Fi monitoring and a safe physical pump-control circuit.

Message to juniors and peers: Start with a small working system, test each block separately and document what you actually implemented. Clear documentation is as important as the code when presenting an engineering project.

2 Introduction

Smart irrigation is an application area where embedded systems can make watering decisions more responsive to soil conditions. A simple controller can monitor a condition, compare it with a predefined limit and operate an actuator only when required.

This project demonstrates that basic control loop using an ESP32, relay and OLED display. The internship sample places the work in the context of industrial exposure and asks students to describe the internship opportunity, design approach, performance testing and learning outcomes.

2.1 About UniConverge Technologies Pvt Ltd

The supplied internship sample states that UniConverge Technologies Pvt Ltd (UCT) was established in 2013 and works in the Digital Transformation domain, with a focus on sustainability and return on investment.

The sample lists technologies including IoT, Cyber Security, cloud computing, Machine Learning, 4G/5G/LoRaWAN communication, Java Full Stack, Python and front-end technologies.

The sample describes UCT Insight as an IoT platform supporting MQTT, CoAP, HTTP, Modbus TCP and OPC UA, with cloud and on-premises deployment options. It also mentions dashboards, analytics and reporting, alerts and notifications, third-party integrations and a rule engine.

The sample describes Factory Watch as a smart-factory platform for production and asset monitoring, OEE, predictive maintenance and scaling toward digital-twin solutions. It also highlights LoRaWAN applications in Agritech, smart cities, industrial monitoring and utility metering.

2.1 About UniConverge Technologies Pvt Ltd (continued)

The sample report also describes UCT's predictive-maintenance work, combining embedded systems, Industrial IoT and machine learning for industrial machine-health monitoring and remaining-useful-life estimation.

2.2 About upskill Campus (USC)

The supplied sample states that upskill Campus, together with The IoT Academy and in association with UniConverge Technologies, facilitated the complete internship process. It describes USC as a career-development platform.

2.3 The IoT Academy

The supplied sample describes The IoT Academy as the EdTech division of UCT and states that it runs long executive certification programs in collaboration with EICT Academy, IIT Kanpur, IIT Roorkee and IIT Guwahati.

2.4 Objectives of this Internship Program

- Gain practical experience of working in the industry.
- Solve real-world problems.
- Improve job prospects.
- Improve understanding of the field and its applications.
- Develop personal skills such as communication and problem solving.

For this project, these objectives were addressed through embedded-system design, Wokwi simulation, ESP32 programming, interface configuration, threshold-based control, testing and technical documentation.

2.5 Reference

[1] Upskill Campus / UCT Sample Internship Report supplied for the internship.

[2] Wokwi project: <https://wokwi.com/projects/473414532081154049>

[3] Arduino Documentation: <https://docs.arduino.cc/>

[4] Espressif ESP32 Documentation: <https://docs.espressif.com/>

[5] Adafruit SSD1306 and GFX documentation: <https://learn.adafruit.com/>

2.6 Glossary

Term	Meaning
ESP32	Microcontroller development platform used as the main controller.
IoT	Internet of Things; connected sensing, processing and control systems.
OLED	Organic Light-Emitting Diode display used to show system status.
I2C	Two-wire serial communication interface used by the OLED.
GPIO	General Purpose Input/Output pin on the ESP32.
Relay	Electrically controlled switch used as the pump-control output.
Threshold	Preset value used to decide whether irrigation should be ON or OFF.
Wokwi	Online electronics simulator used to build and test the prototype.

3 Problem Statement

Manual irrigation depends on a person observing plants and deciding when to water them. Timer-based irrigation can also water plants at fixed times even when the soil condition does not require it. The project addresses this problem by demonstrating a controller that makes an irrigation decision from a soil-condition value.

The target behavior is simple: when the soil is considered dry, the irrigation output should turn ON; when the soil is considered wet, the output should turn OFF. The system should also provide a visible status so that the user can understand the current value and pump state.

Current project scope: The Wokwi circuit contains a potentiometer connected to ESP32 GPIO 34 as the simulated soil input, but the supplied program does not currently read GPIO 34. Instead, it automatically changes the variable soilValue between wet and dry ranges.

4 Existing and Proposed Solution

Existing Solution

Common simple irrigation approaches include manual watering and timer-based watering. Manual watering requires regular attention, while a fixed timer does not directly respond to the current soil condition.

Proposed Solution

The ESP32 runs a threshold-based irrigation controller. The program starts with a soil value of 3000, decreases it by 100 while the simulated soil is drying and then increases it by 100 while the simulated soil is wetting. The threshold is 2000.

Control rule: If soilValue < 2000, GPIO 25 is set LOW and the relay is treated as ON. If soilValue >= 2000, GPIO 25 is set HIGH and the relay is treated as OFF.

Value Addition

The prototype combines sensing logic, decision making, actuation and user feedback in one embedded system. It is repeatable in Wokwi and provides a clear base for later integration with a real sensor, pump and IoT monitoring service.

4.1 Code Submission (GitHub Link)

Repository name: upskillcampus

Code filename: SmartIrrigationSystem.ino

Code link: <https://github.com/prudhvikancharla3/upskillcampus/blob/main/SmartIrrigationSystem.ino>

4.2 Report Submission (GitHub Link)

Report filename: SmartIrrigationSystem_Prudhvi_USC_UCT.pdf

Report link: [Paste the final GitHub report link here after uploading this PDF]

5 Proposed Design / Model

The system is organized as a simple embedded control loop. A soil-condition value is provided to the ESP32. The controller compares the value with the threshold and drives the relay output. The OLED provides local feedback and the Serial Monitor provides a debugging view.

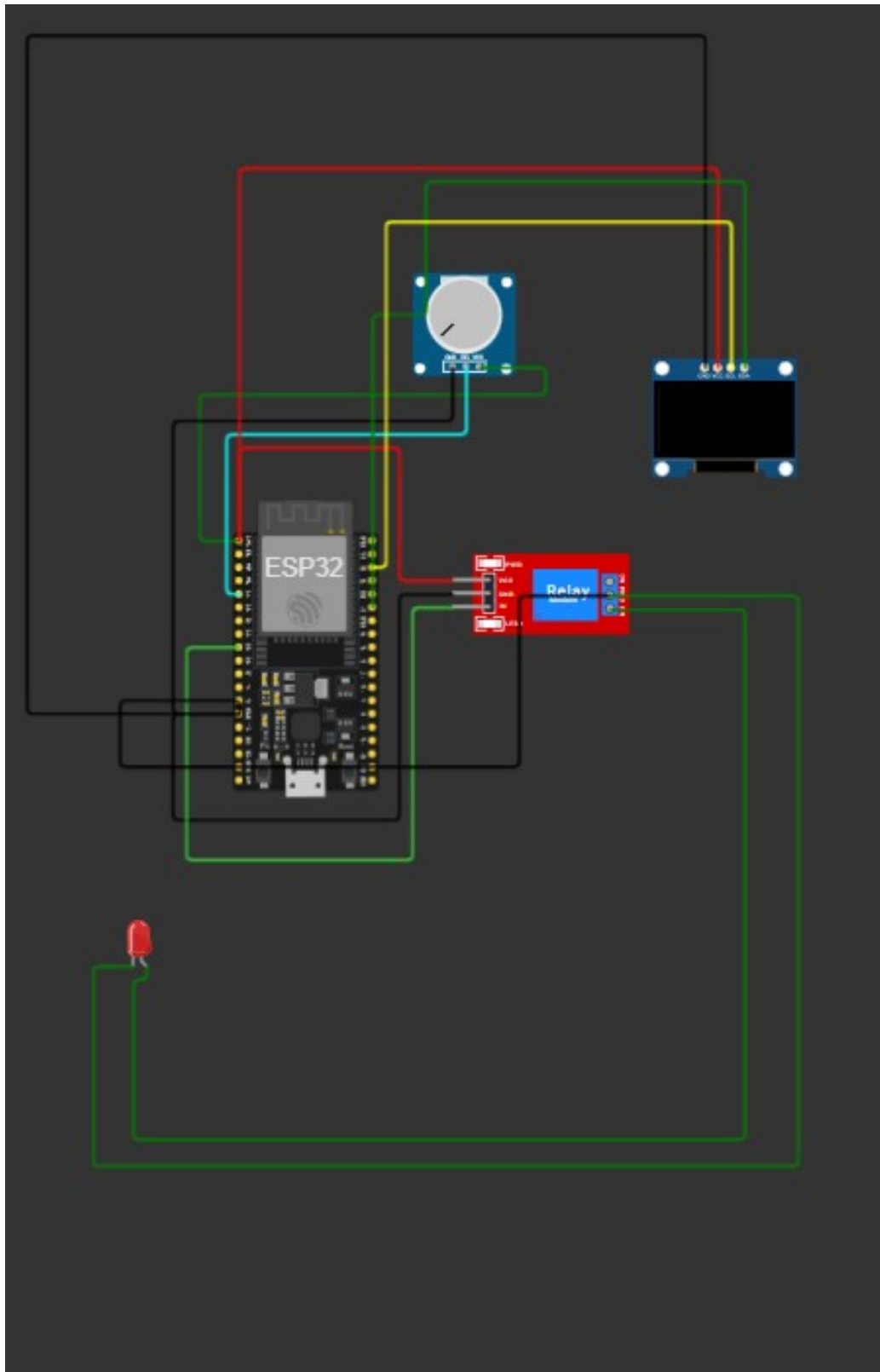


Figure 1: Wokwi circuit used for the Smart Irrigation System.

5.1 High Level Diagram

Stage 1	Stage 2	Stage 3	Stage 4
Soil Condition	ESP32 Controller	Threshold Decision	Relay / Pump
Input value	Processing	Dry / Wet decision	Output control

OLED Display: shows the current soil value and system status. Serial Monitor: displays the simulated soil value for monitoring and debugging.

5.2 Low Level Diagram

Interface	Connection / Setting	Purpose
Soil input	Potentiometer SIG -> GPIO 34	Represents soil-condition input in the Wokwi circuit.
Relay control	Relay IN -> GPIO 25	Controls irrigation output state.
OLED SDA	GPIO 21	I2C data line.
OLED SCL	GPIO 22	I2C clock line.
OLED address	0x3C	SSD1306 I2C device address.
Serial	115200 baud	Displays the soil value for monitoring/debugging.

5.3 Interfaces

The Wokwi diagram connects the potentiometer signal to GPIO 34, the relay input to GPIO 25, OLED SDA to GPIO 21 and OLED SCL to GPIO 22. The OLED uses I2C address 0x3C.

The source code defines GPIO 25 as the relay-control pin and initializes the OLED at 0x3C. The source code does not use `analogRead()` for the potentiometer input; instead, `soilValue` is generated in software.

System Flow

Start -> Initialize ESP32, relay and OLED -> Update soil value -> Compare with threshold 2000 -> Relay ON/OFF -> Update OLED and Serial Monitor -> Delay 1 second -> Repeat.

6 Performance Test

Performance testing is included because the Upskill Campus sample emphasizes why the work should demonstrate practical or industrial relevance rather than only an academic concept.

Identified Constraints

- Decision correctness around the threshold of 2000.
- One-second update interval created by delay(1000).
- Repeatability of dry and wet states.
- Display consistency with the relay decision.
- Simulation-only sensing and actuator behavior.
- Real sensor accuracy, pump flow, power consumption and durability remain untested.

How the constraints were handled

The soil simulation was made automatic so both dry and wet states could be tested repeatedly. Serial Monitor output provides visibility into the soil value, while the OLED confirms the user-facing condition and pump state.

6.1 Test Plan / Test Cases

Test	Input / Condition	Expected Result	Observed Logic
TC01	soilValue = 3000	Wet; Pump OFF	Pass
TC02	soilValue = 2000	Wet; Pump OFF	Pass
TC03	soilValue = 1900	Dry; Pump ON	Pass
TC04	soilValue below 1000	Dry; direction changes later	Pass
TC05	Value rises above 2000	Wet; Pump OFF	Pass
TC06	Repeated drying/wetting cycle	Consistent switching	Pass

6.2 Test Procedure

1. Open the Wokwi project and start the simulation.
2. Observe the startup message on the OLED.
3. Allow the program to reduce the simulated soil value.
4. Verify that the relay state changes to ON below 2000.
5. Observe the OLED and Serial Monitor output.
6. Allow the value to increase and verify relay OFF at 2000 or above.
7. Repeat the cycle and confirm consistent behavior.

6.3 Performance Outcome

The threshold-based control behavior is deterministic and repeatable in simulation. The OLED status follows the same decision used for the relay output. The automatic drying/wetting sequence makes it possible to test both states without manually changing a sensor.

The one-second loop delay provides a simple periodic update. A real irrigation system would require validation of sensor response, plant requirements, electrical safety, pump behavior, power consumption and long-duration operation.

7 My Learnings

The project strengthened my understanding of ESP32 programming and the structure of an embedded application. I learned how setup and loop functions are organized, how GPIO outputs are controlled and how threshold logic can be used to make an automatic decision.

I also learned about OLED display interfacing through I2C, including the use of the Wire library and the SSD1306 display library. Serial Monitor output helped me understand how internal values can be observed while debugging.

Working with Wokwi improved my ability to build and inspect a circuit before moving to physical hardware. The project also showed why the software implementation and circuit diagram must be checked together: the current circuit includes a potentiometer on GPIO 34, while the supplied code simulates the value instead of reading that pin.

From an internship perspective, I learned the importance of testing edge cases, recording limitations honestly and presenting a project in a structured way. These skills will be useful when moving from academic prototypes toward practical embedded and IoT applications.

Technical Skills Improved

- ESP32 programming
- GPIO and relay control
- OLED / I2C interfacing
- Embedded debugging
- Wokwi simulation
- Threshold-based control
- Technical documentation
- Problem-solving and testing

8 Future Work Scope

The current internship implementation demonstrates the control logic using a simulated soil value. The following improvements can be taken up in future versions.

- Replace the software-generated soil value with a calibrated physical soil-moisture sensor and read it through the ESP32 ADC input.
- Add Wi-Fi connectivity for remote monitoring and control.
- Send moisture and pump data to an IoT or cloud dashboard.
- Add adjustable moisture thresholds instead of a fixed value of 2000.
- Add rain detection and water-tank level sensing.
- Add pump dry-run protection and flow monitoring.
- Add alerts for abnormal moisture, empty tank or pump faults.
- Support multiple irrigation zones.
- Measure power consumption and validate relay/pump durability.
- Build a safe physical prototype with appropriate isolation, power supply and protection.

Conclusion

The Smart Irrigation System demonstrates a complete embedded control concept using ESP32, threshold-based decision making, relay control and OLED feedback. The Wokwi simulation provides a repeatable environment for observing dry and wet operating states.

The project covers the main areas expected in the supplied Upskill Campus report structure: problem definition, proposed solution, design, interfaces, performance testing, learning outcomes and future scope.

The most important limitation is that the present program simulates the soil value instead of reading the potentiometer or a physical soil sensor. The next development step is therefore to connect the actual input to the control algorithm and validate the complete system with a real pump and safe power-control hardware.

Appendix - Complete Source Code

The following is the source code contained in the submitted Wokwi project. The source uses the Adafruit GFX and Adafruit SSD1306 libraries.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

#define RELAY_PIN 25
#define THRESHOLD 2000

int soilValue = 3000;    // Start as wet
bool drying = true;      // Direction of change

void setup() {
  Serial.begin(115200);
  pinMode(RELAY_PIN, OUTPUT);

  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("SSD1306 allocation failed"));
    for(;;);
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);
  display.println("Hello Rancher!");
  display.println("");
  display.println("Monitoring soil moisture...");
  display.display();
  delay(3000);
}

void loop() {
  // Simulate soil drying/wetting automatically
  if (drying) {
    soilValue -= 100;    // soil dries
    if (soilValue < 1000) drying = false; // switch to wetting
  } else {
    soilValue += 100;    // soil gets wet
    if (soilValue > 3000) drying = true;  // switch to drying
  }

  Serial.println(soilValue);

  display.clearDisplay();
  display.setCursor(0, 0);
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);

  display.println("Soil Value: " + String(soilValue));
  display.println("");

  if (soilValue < THRESHOLD) {
    digitalWrite(RELAY_PIN, LOW); // Relay ON
    display.println("💧 Soil is Dry");
    display.println("");
    display.println("Pump: ON ✅");
    display.println("");
    display.println("Pumping water...");
  } else {
    digitalWrite(RELAY_PIN, HIGH); // Relay OFF
    display.println("🌱 Soil is Wet");
    display.println("");
    display.println("Pump: OFF ❌");
    display.println("");
    display.println("Saving water...");
  }

  display.display();
  delay(1000);
}
```

Required Libraries

Wokwi Library List
See <https://docs.wokwi.com/guides/libraries>

Automatically added based on includes:

Adafruit GFX Library
Adafruit SSD1306